

TLB Design and Performance

Translation Lookaside Buffer, Address Translation and Effective Access Time

Prashant Jadav

DEPSTAR - CHARUSAT)

Learning Objectives

- Explain why a Translation Lookaside Buffer (TLB) is needed.
- Describe TLB entries, tags, VPNs, PFNs and protection bits.
- Explain TLB hit, TLB miss and the translation path.
- Calculate Effective Access Time (EAT).
- Analyze the effect of TLB hit ratio on performance.
- Calculate TLB tag, set-index and offset bits.
- Evaluate TLB design choices such as associativity, size and replacement.
- Solve GATE-level TLB and EAT numericals.

Why Do We Need a TLB?

The problem

In a paged virtual-memory system:

Virtual Page Number (VPN) $\longrightarrow$ Physical Frame Number (PFN)

Without a TLB:

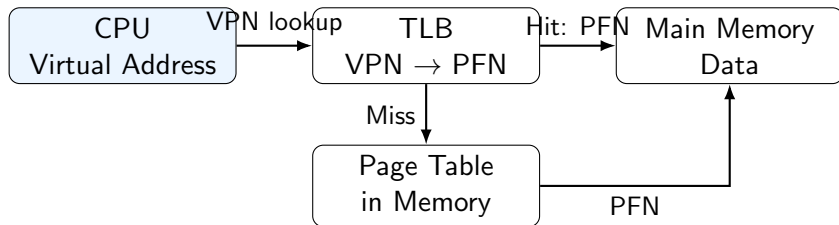
Page-table access + data access

With a TLB hit:

TLB lookup + data access

- The page table is usually much larger than the TLB.
- The TLB stores recently used virtual-to-physical translations.
- Locality of reference makes TLB caching effective.

Basic Address Translation Path



- **TLB hit:** translation found in TLB.
- **TLB miss:** page table must be consulted.
- Standard no-page-fault model: a TLB miss requires one extra memory access for the PTE.

Anatomy of a TLB Entry

Field	Purpose
VPN / Tag	Identifies the virtual page
PFN	Physical frame number corresponding to the VPN
Valid bit	Indicates whether the entry is usable
Protection bits	Read/write/execute permissions
ASID / PCID	Optional address-space identifier
Replacement/status bits	Optional hardware-management state

Key idea

A TLB caches *translations*, not complete pages.

TLB Hit vs. TLB Miss

Let

T_T = TLB lookup time, T_M = one main-memory access

TLB Hit

$$T_{\text{hit}} = T_T + T_M$$

TLB Miss – single-level page table

$$T_{\text{miss}} = T_T + T_M + T_M = T_T + 2T_M$$

If the TLB hit ratio is h :

$$EAT = h(T_T + T_M) + (1 - h)(T_T + 2T_M)$$

Hence:

$$EAT = T_T + T_M(2 - h)$$

Worked Example: Basic EAT

Given:

$$T_T = 10 \text{ ns}, \quad T_M = 100 \text{ ns}, \quad h = 0.90$$

$$EAT = 0.9(10 + 100) + 0.1(10 + 200)$$

$$= 0.9(110) + 0.1(210)$$

$$= 99 + 21$$

$$= \boxed{120 \text{ ns}}$$

Without a TLB:

$$T_{\text{no TLB}} = 2T_M = 200 \text{ ns}$$

$$\frac{200}{120} \approx 1.67\times$$

Exam shortcut

For the standard single-level page-table model:

$$\boxed{EAT = T_T + T_M(2 - h)}$$

TLB Performance Metrics

- **TLB hit ratio (h):** fraction of translations found in TLB.
- **TLB miss ratio:** $1 - h$.
- **TLB lookup latency:** time to search the TLB.
- **TLB reach:**

entries $\times$ page size

- **EAT:** average time per memory reference.

Design intuition

More entries or larger pages can increase TLB reach, but may introduce hardware, memory-fragmentation or page-transfer trade-offs.

TLB Organization and Associativity

Common organizations:

- 1 Fully associative TLB
- 2 Direct-mapped TLB
- 3 Set-associative TLB

For an N -entry TLB with A ways:

$$\text{Number of sets} = \frac{N}{A}$$

If there are 2^s sets:

$$\text{Set-index bits} = s$$

For a V -bit virtual address and page size 2^p bytes:

$$\text{VPN bits} = V - p$$

Without an address-space identifier:

$$\text{Tag bits} = V - p - s$$

GATE Numerical 1: TLB EAT

GATE IT 2008, Question 16 – paraphrased.

Given:

$$T_T = 10 \text{ ns}, \quad T_M = 50 \text{ ns}, \quad h = 0.90$$

No page faults. Find EAT.

$$\begin{aligned} EAT &= 0.9(10 + 50) + 0.1(10 + 50 + 50) \\ &= 54 + 11 \\ &= \boxed{65 \text{ ns}} \end{aligned}$$

Answer: 65 ns

GATE Numerical 2: TLB EAT

GATE CSE 2014, Set 3, Question 33 – paraphrased.

Given:

$$T_T = 10 \text{ ms}, \quad T_M = 80 \text{ ms}, \quad h = 0.60$$

Assume page table and pages are in physical memory.

$$\begin{aligned} EAT &= 0.6(10 + 80) + 0.4(10 + 80 + 80) \\ &= 54 + 68 \\ &= \boxed{122 \text{ ms}} \end{aligned}$$

Answer: 122 ms

Common mistake

On a TLB miss, include the memory access used to obtain the page-table entry.

GATE Numerical 3: TLB Tag Bits

GATE CSE 2015, Set 2 – paraphrased.

Given:

$V = 40$ -bit virtual address

Page size = $8\text{ KB} = 2^{13}$ bytes

TLB = 128 entries, 32 sets, 4 ways

Page offset = 13, Set index = $\log_2 32 = 5$

Therefore:

$$\text{Tag} = 40 - 13 - 5 = 22 \text{ bits}$$

Answer: 22 bits

GATE Numerical 4: TLB + Page Fault

GATE CSE 2020, Question 53 – paraphrased.

Given:

$$T_M = 100 \text{ ns}, \quad T_T = 20 \text{ ns}, \quad T_{\text{page}} = 5000 \text{ ns}$$

$$h = 0.95, \quad p_f = 0.10, \quad 20\% \text{ of page faults require a dirty-page write-back}$$

Using the interpretation associated with the keyed solution:

$$\begin{aligned} EAT &= 0.95(20 + 100) \\ &\quad + 0.05\{0.90(20 + 100 + 100) \\ &\quad + 0.10[0.80(20 + 100 + 5000 + 100) \\ &\quad + 0.20(20 + 100 + 5000 + 5000 + 100)]\} \end{aligned}$$

$$\boxed{EAT = 155.0 \text{ ns}}$$

Why TLB Hit Ratio Matters

For the standard single-level case:

$$EAT = T_T + T_M(2 - h)$$

Suppose $T_T = 10\text{ ns}$ and $T_M = 100\text{ ns}$.

TLB hit ratio	EAT
0.50	160 ns
0.75	135 ns
0.90	120 ns
0.95	115 ns
0.99	111 ns

Observation

A higher hit ratio reduces the average cost because a TLB hit avoids a page-table memory access.

TLB Design Trade-offs

Design choice	Benefit	Cost / Trade-off
More entries	Higher reach, fewer misses	Larger hardware
Higher associativity	Fewer conflict misses	More parallel comparisons
Larger page size	Higher reach	Fragmentation/page-transfer trade-offs
Longer tags	More address discrimination	More comparator/storage bits
ASID/PCID	Fewer context-switch flushes	Extra tag/state bits
Better replacement	Better locality	Replacement metadata overhead

$$\text{TLB Reach} = \text{TLB entries} \times \text{page size}$$

Example:

$$512 \times 4 \text{ KB} = 2048 \text{ KB} = 2 \text{ MB}$$

If the active translation working set is much larger than the TLB reach, TLB misses may increase.

Exam distinction

TLB reach is the virtual-address space whose translations can be cached simultaneously; it is not the physical-memory capacity.

Multi-level Page Tables and TLBs

For an L -level page table:

$$T_{\text{miss}} \approx T_T + (L + 1)T_M$$

For a TLB hit:

$$T_{\text{hit}} \approx T_T + T_M$$

Thus:

$$EAT = h(T_T + T_M) + (1 - h)[T_T + (L + 1)T_M]$$

Design implication

As page-table depth increases, a high TLB hit ratio becomes even more important for performance.

Exam-Solving Procedure

- 1 Identify T_T , T_M and hit ratio h .
- 2 Determine the number of page-table levels.
- 3 Draw hit and miss paths.
- 4 Count memory accesses on each path.
- 5 Check whether page faults are included.
- 6 Convert all time units to the same unit.
- 7 Compute the weighted average.
- 8 For tag questions, calculate page-offset and set-index bits first.

Common errors

Forgetting the page-table access on a TLB miss; mixing ns and ms; using hit ratio instead of miss ratio; ignoring stated page-fault assumptions.

Practice Numerical 1

Given:

$$T_T = 5 \text{ ns}, \quad T_M = 80 \text{ ns}, \quad h = 0.95$$

Single-level page table, no page faults.

$$\begin{aligned} EAT &= 0.95(5 + 80) + 0.05(5 + 80 + 80) \\ &= 0.95(85) + 0.05(165) = \boxed{89 \text{ ns}} \end{aligned}$$

Practice Numerical 2: Tag Calculation

A TLB has 256 entries, 64 sets and 4 ways. The virtual address is 48 bits and page size is 4 KB.

$$\text{Offset} = \log_2(4096) = 12$$

$$\text{Index} = \log_2(64) = 6$$

$$\text{Tag} = 48 - 12 - 6 = 30 \text{ bits}$$

Practice Numerical 3: TLB Reach

A TLB contains 1024 entries and uses 8 KB pages.

$$\text{TLB Reach} = 1024 \times 8 \text{ KB} = 8192 \text{ KB} = \boxed{8 \text{ MB}}$$

If the active translation working set is 32 MB, explain why TLB misses may become frequent.

Answer

The TLB can simultaneously cache translations for about 8 MB. A much larger active translation working set creates translation-cache pressure.

Key Takeaways

- TLB is a cache of recent virtual-to-physical translations.
- TLB hit avoids a page-table memory access.
- Single-level standard model:

$$EAT = T_T + T_M(2 - h)$$

- TLB reach depends on entries and page size.
- Set-associative TLBs require set-index and tag calculations.
- Higher hit ratio generally improves EAT.
- GATE frequently tests EAT, tag bits, TLB organization and page-fault interactions.

Remember: Count the memory accesses on every path.